Amazon S3 Fleet
LLD Interview Guide

5 high-priority design questions with simple Python implementations

Prepared for interview practice

How this guide is structured

For every question: clarification communication -> main entities -> entity functions -> best design pattern and reasoning -> simple runnable-style Python implementation.

Important

These are preparation questions chosen for strong relevance to infrastructure, automation, storage, eventing, and maintainable-code interviews. They are not a claim that Amazon will ask these exact questions.

Contents

1. Task / Fleet Scheduler - Priority, retries, task state, extensibility

2. Notification System - Email/SMS/push with pluggable channels

3. Event Forwarding System - Publish/subscribe event handlers

4. Log & Monitoring System - Ingest logs and query with composable filters

5. S3-like Object Storage - Buckets, objects, multipart/resumable uploads

Recommended interview rhythm

- Spend the first 2-4 minutes clarifying scope instead of immediately coding.
- Name the entities and their responsibilities before writing classes.
- State one main design pattern and explain exactly what can vary.
- Implement the smallest clean version first. Add retries, concurrency, persistence, or scale only when the interviewer asks.
- While coding, keep responsibilities separate and make extension points obvious.

1. Design a Task / Fleet Scheduler

Typical interview prompt: Design a system that schedules tasks to run on workers or machines. Tasks can have priorities, can fail, and may be retried.

A. Clarification communication

Candidate: Before I design it, I want to confirm the scope. Are tasks submitted dynamically while the scheduler is running?

Interviewer: Yes.

Candidate: Should higher-priority tasks run before lower-priority tasks?

Interviewer: Yes.

Candidate: Can a task fail, and if it fails should we retry it?

Interviewer: Yes, allow a small retry limit.

Candidate: Do I need distributed persistence or multiple scheduler nodes, or should I focus on clean in-memory LLD?

Interviewer: Focus on the in-memory design first.

Candidate: Great. I will design an in-memory scheduler with a pluggable scheduling strategy, task states, workers, and bounded retries. I will keep persistence and multi-node coordination as extensions.

Finalized scope

submit task, choose next task by priority, execute on a worker, track status, retry failures up to a limit, and allow the scheduling algorithm to be replaced later.

B. Main entities

Entity	Responsibility	Important functions
Task	Represents one unit of work and its current state.	run(), id, priority, status, retries
SchedulingStrategy	Defines how the next task is chosen.	add(task), next_task()
PrioritySchedulingStrategy	Concrete strategy using a priority queue.	add(task), next_task()
Worker	Executes one task.	execute(task)
Scheduler	Coordinates submission, selection, execution, and retry.	submit(task), run_next()

C. Best design pattern: Strategy

- What varies: the rule used to choose the next task - FIFO, priority, shortest-job-first, deadline-based, etc.
- Why Strategy fits: Scheduler should depend on a small interface instead of hard-coding one queue policy.
- Benefit: adding a FIFO or deadline strategy does not require changing Scheduler.
- Task status is represented with an enum. A full State pattern is unnecessary unless each state develops complex behavior.

Interview sentence

"I am using Strategy because scheduling policy is the behavior most likely to change. Scheduler owns orchestration; the strategy owns task ordering."

D. Simple Python implementation

```
from abc import ABC, abstractmethod
from dataclasses import dataclass
from enum import Enum
import heapq
import itertools
```

```
class TaskStatus(Enum):
    PENDING = "PENDING"
    RUNNING = "RUNNING"
    SUCCESS = "SUCCESS"
    FAILED = "FAILED"


@dataclass
class Task:
    task_id: str
    priority: int
    should_fail: bool = False
    max_retries: int = 2
    retries: int = 0
    status: TaskStatus = TaskStatus.PENDING

    def run(self):
        """Real code would call the actual job logic here."""
        if self.should_fail:
            raise RuntimeError("Task failed")
        print(f"Running {self.task_id}")


class SchedulingStrategy(ABC):
    @abstractmethod
    def add(self, task: Task):
        pass

    @abstractmethod
    def next_task(self):
        pass


class PrioritySchedulingStrategy(SchedulingStrategy):
    def __init__(self):
        self.heap = []
        self.counter = itertools.count()

    def add(self, task: Task):
        # Python heap is a min-heap. Negative makes larger priority win.
        heapq.heappush(
            self.heap,
            (-task.priority, next(self.counter), task),
        )

    def next_task(self):
        if not self.heap:
            return None
        return heapq.heappop(self.heap)[2]


class Worker:
    def execute(self, task: Task):
        task.status = TaskStatus.RUNNING
        task.run()
        task.status = TaskStatus.SUCCESS


class Scheduler:
    def __init__(self, strategy: SchedulingStrategy):
        self.strategy = strategy
        self.worker = Worker()

    def submit(self, task: Task):
        task.status = TaskStatus.PENDING
        self.strategy.add(task)

    def run_next(self):
        task = self.strategy.next_task()
        if task is None:
            return None

        try:
            self.worker.execute(task)
        except Exception:
            task.retries += 1

            if task.retries <= task.max_retries:
                task.status = TaskStatus.PENDING
                self.strategy.add(task)
```

```
            else:
                task.status = TaskStatus.FAILED

        return task


# Example
scheduler = Scheduler(PrioritySchedulingStrategy())
scheduler.submit(Task("health-check", priority=5))
scheduler.submit(Task("critical-repair", priority=10))

scheduler.run_next()   # critical-repair first
scheduler.run_next()   # health-check second
```

E. What to say about extensions

- Multiple workers: keep a worker pool and assign tasks only to available workers.
- Delayed tasks: add execute_at and order by time before priority.
- Thread safety: protect shared queue/state with a lock or use a thread-safe queue.
- Distributed version: persist task state and use leases/heartbeats so two schedulers do not execute the same task.
- Retry policy: extract retry behavior into another Strategy if retry logic becomes complex.

2. Design a Notification System

Typical interview prompt: Design a notification system that can send the same notification through email, SMS, and push. New channels should be easy to add.

A. Clarification communication

Candidate: Can one notification be sent through multiple channels?

Interviewer: Yes.

Candidate: Should I integrate real third-party email/SMS providers?

Interviewer: No, just model the channel behavior.

Candidate: Do we need to track send status?

Interviewer: Yes, a simple sent/failed result is enough.

Candidate: Should adding a new channel such as Slack require changes to the service?

Interviewer: Ideally no.

Candidate: I will define a common channel interface, implement Email/SMS/Push strategies, and let NotificationService depend only on that abstraction.

Finalized scope

send one notification to one user through one or more channels, keep channel-specific behavior separate, and make new channels easy to add.

B. Main entities

Entity	Responsibility	Important functions
User	Recipient contact information.	user_id, email, phone, device_token
Notification	Message content to send.	notification_id, subject, body
NotificationChannel	Common interface for delivery behavior.	send(user, notification)
Email/SMS/PushChannel	Concrete delivery strategies.	send(...)
NotificationService	Coordinates sending through selected channels.	send(user, notification, channels)

C. Best design pattern: Strategy

- What varies: the delivery algorithm for each channel.
- Every channel implements the same send() contract.
- NotificationService does not contain if/elif logic for Email, SMS, Push, Slack, etc.
- This follows Open/Closed Principle: add a new channel class without modifying the orchestration logic.

Interview sentence

"Email, SMS, and push are interchangeable ways to perform the same operation, so Strategy is the cleanest fit."

D. Simple Python implementation

```
from abc import ABC, abstractmethod
from dataclasses import dataclass


@dataclass
class User:
    user_id: str
    email: str | None = None
    phone: str | None = None
    device_token: str | None = None
```

```
@dataclass
class Notification:
    notification_id: str
    subject: str
    body: str


class NotificationChannel(ABC):
    @abstractmethod
    def send(self, user: User, notification: Notification) -> bool:
        pass


class EmailChannel(NotificationChannel):
    def send(self, user, notification):
        if not user.email:
            return False
        print(f"EMAIL to {user.email}: {notification.body}")
        return True


class SMSChannel(NotificationChannel):
    def send(self, user, notification):
        if not user.phone:
            return False
        print(f"SMS to {user.phone}: {notification.body}")
        return True


class PushChannel(NotificationChannel):
    def send(self, user, notification):
        if not user.device_token:
            return False
        print(f"PUSH to {user.device_token}: {notification.body}")
        return True


class NotificationService:
    def send(self, user, notification, channels):
        results = {}

        for channel in channels:
            name = channel.__class__.__name__
            try:
                results[name] = channel.send(user, notification)
            except Exception:
                results[name] = False

        return results


# Example
user = User(
    user_id="u1",
    email="user@example.com",
    phone="555-0100",
)
notification = Notification("n1", "Alert", "Host is unhealthy")

service = NotificationService()
result = service.send(
    user,
    notification,
    [EmailChannel(), SMSChannel()],
)

print(result)
```

E. What to say about extensions

- Retries: wrap a channel with a retry policy or queue failed deliveries.
- Factory: useful if callers provide channel names such as "email" instead of channel objects.
- Preferences: UserPreference can decide which channels are enabled.
- Templates: introduce TemplateRenderer Strategy if each channel needs different message formatting.
- Async delivery: NotificationService writes jobs to a queue; workers call the channel strategies.

3. Design an Event Forwarding System

Typical interview prompt: Design a framework where producers publish events and multiple consumers can subscribe to and process those events.

A. Clarification communication

Candidate: Can multiple consumers subscribe to the same event type?

Interviewer: Yes.

Candidate: Should a producer know which consumers receive the event?

Interviewer: No.

Candidate: If one consumer fails, should the other consumers still receive the event?

Interviewer: Yes.

Candidate: Do you want an in-memory framework first or a distributed Kafka-like system?

Interviewer: Start in memory.

Candidate: I will use publish/subscribe. Producers publish to an EventBus; handlers subscribe by event type; the bus forwards independently to each handler.

Finalized scope

publish typed events, allow many subscribers per type, keep publishers decoupled from subscribers, and isolate handler failures.

B. Main entities

Entity	Responsibility	Important functions
Event	Immutable data describing something that happened.	event_type, payload
EventHandler	Common consumer interface.	handle(event)
Concrete handlers	Perform different reactions to the same event.	handle(event)
EventBus	Stores subscriptions and forwards published events.	subscribe(type, handler), publish(event)
Producer	Creates events and publishes them.	publish(...) via EventBus

C. Best design pattern: Observer

- The EventBus is the subject/publisher.
- Handlers are observers/subscribers.
- A publisher does not need to know how many consumers exist or what they do.
- Adding a new subscriber does not change producer code.

Interview sentence

"This is a direct Observer / publish-subscribe problem: one event can fan out to many independent listeners."

D. Simple Python implementation

```
from abc import ABC, abstractmethod
from dataclasses import dataclass
from collections import defaultdict


@dataclass(frozen=True)
class Event:
    event_type: str
    payload: dict
```

```
class EventHandler(ABC):
    @abstractmethod
    def handle(self, event: Event):
        pass


class MonitoringHandler(EventHandler):
    def handle(self, event):
        print("Monitoring:", event.payload)


class RepairHandler(EventHandler):
    def handle(self, event):
        print("Repair requested for:", event.payload.get("host_id"))


class AlertHandler(EventHandler):
    def handle(self, event):
        print("Alert sent for:", event.payload.get("host_id"))


class EventBus:
    def __init__(self):
        self.subscribers = defaultdict(list)

    def subscribe(self, event_type: str, handler: EventHandler):
        self.subscribers[event_type].append(handler)

    def publish(self, event: Event):
        for handler in self.subscribers[event.event_type]:
            try:
                handler.handle(event)
            except Exception as exc:
                # One broken consumer should not stop the others.
                print("Handler failed:", exc)


# Example
bus = EventBus()
bus.subscribe("HOST_UNHEALTHY", MonitoringHandler())
bus.subscribe("HOST_UNHEALTHY", RepairHandler())
bus.subscribe("HOST_UNHEALTHY", AlertHandler())

bus.publish(
    Event(
        event_type="HOST_UNHEALTHY",
        payload={"host_id": "host-17", "reason": "disk error"},
    )
)
```

E. What to say about extensions

- Asynchronous processing: put events onto queues and let subscriber workers consume independently.
- Retry / dead-letter queue: failed events are retried and eventually moved aside for investigation.
- Batch consumers: let handlers request a list of events instead of one event at a time.
- Ordering: preserve order within a partition/key such as host_id.
- Backpressure: limit queue size or consumer concurrency so slow consumers do not exhaust memory.

4. Design a Log & Monitoring System

Typical interview prompt: Design an in-memory system that accepts logs from many hosts and lets callers search using filters such as severity, host, or time.

A. Clarification communication

Candidate: What fields should each log contain?

Interviewer: Timestamp, host, severity, and message are enough.

Candidate: Should users be able to combine filters, for example host = A AND severity = ERROR?

Interviewer: Yes.

Candidate: Do I need persistence or distributed search?

Interviewer: No, focus on the object design.

Candidate: Should adding a new filter require changing LogStore?

Interviewer: No.

Candidate: Then I will keep storage simple and model filtering as independent interchangeable objects that implement matches(log).

Finalized scope

ingest log entries, store them in memory, query using reusable filters, and compose filters without hard-coding every query combination.

B. Main entities

Entity	Responsibility	Important functions
LogEntry	Represents one log line.	timestamp, host_id, level, message
LogFilter	Common filtering contract.	matches(log)
HostFilter	Matches one host.	matches(log)
LevelFilter	Matches a severity level.	matches(log)
AndFilter	Combines multiple filters.	matches(log)
LogStore	Stores and retrieves logs.	add(log), search(filter)

C. Best design pattern: Strategy / Specification

- Each filter encapsulates one matching rule behind the same matches() interface.
- LogStore depends only on LogFilter, so new filters do not change storage code.
- AndFilter composes filters. OR/NOT filters can be added the same way.
- In an interview, calling this Strategy is completely reasonable; more specifically, composable query rules resemble the Specification pattern.

Interview sentence

"The filtering rule is the changing behavior. I will encapsulate it behind a LogFilter interface so LogStore stays closed to query-rule changes."

D. Simple Python implementation

```
from abc import ABC, abstractmethod
from dataclasses import dataclass
from datetime import datetime
from enum import Enum


class LogLevel(Enum):
    INFO = 1
```

```
    WARN = 2
    ERROR = 3


@dataclass(frozen=True)
class LogEntry:
    timestamp: datetime
    host_id: str
    level: LogLevel
    message: str


class LogFilter(ABC):
    @abstractmethod
    def matches(self, log: LogEntry) -> bool:
        pass


class HostFilter(LogFilter):
    def __init__(self, host_id: str):
        self.host_id = host_id

    def matches(self, log):
        return log.host_id == self.host_id


class LevelFilter(LogFilter):
    def __init__(self, minimum_level: LogLevel):
        self.minimum_level = minimum_level

    def matches(self, log):
        return log.level.value >= self.minimum_level.value


class AndFilter(LogFilter):
    def __init__(self, *filters):
        self.filters = filters

    def matches(self, log):
        return all(f.matches(log) for f in self.filters)


class LogStore:
    def __init__(self):
        self.logs = []

    def add(self, log: LogEntry):
        self.logs.append(log)

    def search(self, log_filter: LogFilter):
        return [log for log in self.logs if log_filter.matches(log)]


# Example
store = LogStore()
store.add(LogEntry(datetime.now(), "host-1", LogLevel.INFO, "started"))
store.add(LogEntry(datetime.now(), "host-1", LogLevel.ERROR, "disk failed"))
store.add(LogEntry(datetime.now(), "host-2", LogLevel.ERROR, "timeout"))

query = AndFilter(
    HostFilter("host-1"),
    LevelFilter(LogLevel.ERROR),
)

for log in store.search(query):
    print(log)
```

E. What to say about extensions

- TimeRangeFilter, MessageContainsFilter, OrFilter, and NotFilter are easy additions.
- Indexes: maintain dictionaries by host or severity when scanning all logs becomes too slow.
- Retention: evict logs older than a TTL.
- Monitoring rules: subscribe alert handlers to ERROR logs or threshold breaches.
- Distributed version: partition by time/host and store logs in a scalable search backend.

5. Design an S3-like Object Storage System

Typical interview prompt: Design a simplified object storage service with buckets, put/get/delete, and resumable multipart uploads.

A. Clarification communication

Candidate: Should I model object storage rather than a hierarchical file system?

Interviewer: Yes, buckets and object keys are enough.

Candidate: Should basic operations be create bucket, put, get, and delete object?

Interviewer: Yes.

Candidate: For large uploads, should the client upload independent parts and resume only the missing parts after a failure?

Interviewer: Yes.

Candidate: Do I need replication, consistency, authorization, or distributed persistence now?

Interviewer: No, focus on clean LLD first.

Candidate: I will keep an in-memory backend, model multipart uploads with an UploadSession, and hide storage mechanics behind a StorageBackend abstraction.

Finalized scope

create buckets, store/get/delete objects, start multipart upload, upload parts independently, and complete the upload by joining parts in order.

B. Main entities

Entity	Responsibility	Important functions
StoredObject	Represents stored bytes plus key.	key, data
Bucket	Namespace containing objects.	name, objects conceptually
UploadSession	Tracks parts of an unfinished multipart upload.	upload_id, bucket, key, parts
StorageBackend	Abstracts how final objects are persisted.	put(), get(), delete()
InMemoryBackend	Simple storage implementation for interview code.	put(), get(), delete()
ObjectStorageService	Public API and multipart coordinator.	create_bucket(), put_object(), start_upload(), upload_part(), complete_upload()

C. Best design pattern: Strategy

- What varies: where/how bytes are physically stored - memory, local disk, database, distributed storage, etc.
- ObjectStorageService talks only to StorageBackend.
- The multipart workflow stays the same even if the physical storage implementation changes.
- UploadSession is a domain object, not a design pattern; it exists so partial progress can survive a retry in a real persistent implementation.

Interview sentence

"I am separating the object API from the storage backend using Strategy. That keeps the service testable and lets the backend change without rewriting multipart logic."

D. Simple Python implementation

```
from abc import ABC, abstractmethod
from dataclasses import dataclass, field
```

```
import uuid


@dataclass
class StoredObject:
    key: str
    data: bytes


@dataclass
class UploadSession:
    upload_id: str
    bucket: str
    key: str
    parts: dict[int, bytes] = field(default_factory=dict)


class StorageBackend(ABC):
    @abstractmethod
    def put(self, bucket: str, key: str, data: bytes):
        pass

    @abstractmethod
    def get(self, bucket: str, key: str) -> bytes:
        pass

    @abstractmethod
    def delete(self, bucket: str, key: str):
        pass


class InMemoryBackend(StorageBackend):
    def __init__(self):
        self.data = {}

    def put(self, bucket, key, data):
        self.data[(bucket, key)] = data

    def get(self, bucket, key):
        return self.data[(bucket, key)]

    def delete(self, bucket, key):
        self.data.pop((bucket, key), None)


class ObjectStorageService:
    def __init__(self, backend: StorageBackend):
        self.backend = backend
        self.buckets = set()
        self.uploads = {}

    def create_bucket(self, bucket: str):
        self.buckets.add(bucket)

    def _require_bucket(self, bucket: str):
        if bucket not in self.buckets:
            raise ValueError("Bucket does not exist")

    def put_object(self, bucket: str, key: str, data: bytes):
        self._require_bucket(bucket)
        self.backend.put(bucket, key, data)

    def get_object(self, bucket: str, key: str) -> bytes:
        self._require_bucket(bucket)
        return self.backend.get(bucket, key)

    def delete_object(self, bucket: str, key: str):
        self._require_bucket(bucket)
        self.backend.delete(bucket, key)

    def start_multipart_upload(self, bucket: str, key: str) -> str:
        self._require_bucket(bucket)
        upload_id = str(uuid.uuid4())
        self.uploads[upload_id] = UploadSession(upload_id, bucket, key)
        return upload_id

    def upload_part(self, upload_id: str, part_number: int, data: bytes):
        session = self.uploads[upload_id]
        # Re-uploading a part simply replaces it -> retry friendly.
        session.parts[part_number] = data
```

```
    def complete_upload(self, upload_id: str):
        session = self.uploads[upload_id]
        ordered_parts = [
            session.parts[n]
            for n in sorted(session.parts)
        ]
        final_data = b"".join(ordered_parts)
        self.backend.put(session.bucket, session.key, final_data)
        del self.uploads[upload_id]


# Example
service = ObjectStorageService(InMemoryBackend())
service.create_bucket("photos")

upload_id = service.start_multipart_upload("photos", "trip.jpg")
service.upload_part(upload_id, 1, b"AAA")
service.upload_part(upload_id, 2, b"BBB")

# If part 2 failed, client could upload part 2 again without restarting part 1.
service.complete_upload(upload_id)

print(service.get_object("photos", "trip.jpg"))  # b'AAABBB'
```

E. What to say about extensions

- Checksums: store a checksum per part and validate corrupted retries.
- List uploaded parts: client asks which parts already exist before resuming.
- Persistence: store UploadSession metadata so process restarts do not lose progress.
- Versioning: map one key to multiple object versions.
- Large-scale system: partition metadata, replicate object data, authorize requests, and garbage-collect abandoned uploads.

One-page Pattern Cheat Sheet

Question	Main pattern	One-line reason
Task Scheduler	Strategy	Scheduling policy changes: priority, FIFO, deadline, etc.
Notification System	Strategy	Delivery mechanism changes: email, SMS, push, Slack.
Event Forwarding	Observer	One publisher fans out events to many independent subscribers.
Log & Monitoring	Strategy / Specification	Query/filter rules change and need composition.
S3-like Storage	Strategy	Physical storage backend changes while API/workflow remains stable.

Fastest answer structure to memorize

- 1. Clarify scope and state assumptions.
- 2. Name 4-6 entities and give each exactly one responsibility.
- 3. Explain the main relationship between those entities.
- 4. Name one pattern and say what behavior is expected to change.
- 5. Code interfaces first, then concrete classes, then the coordinating service.
- 6. Finish with 2-3 extensions only if time remains.

Key principle

Do not force design patterns into the answer. Use a pattern only when you can clearly state what varies and why the abstraction makes the design easier to extend.